Project Management with Git

Program 5

Collaboration and Remote Repositories:

Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.

1. Fetch the latest changes from the remote repository:

git fetch

This command fetches the latest changes from the remote repository without automatically merging them into your local branches.

2. Rebase your local branch onto the updated remote branch:

Assuming you are currently on the branch you want to update (replace your-branch with the actual name of your branch):

git rebase origin/your-branch

This command applies your local commits on top of the changes fetched from the remote branch. If

conflicts arise, Git will pause the rebase process and ask you to resolve them. Alternatively, you can use

the interactive rebase to review and modify commits during the rebase:

git rebase -i origin/your-branch

This opens an editor where you can pick, squash, or edit individual commits.

3. Continue the rebase or resolve conflicts:

If conflicts occurred during the rebase, Git will prompt you to resolve them. After resolving conflicts, you can continue the rebase with:

git rebase –continue

If you decide to abort the rebase at any point, you can use:

git rebase –abort

4. Push the rebased branch to the remote repository:

After successfully rebasing your local branch, you may need to force-push the changes to the remote

repository:

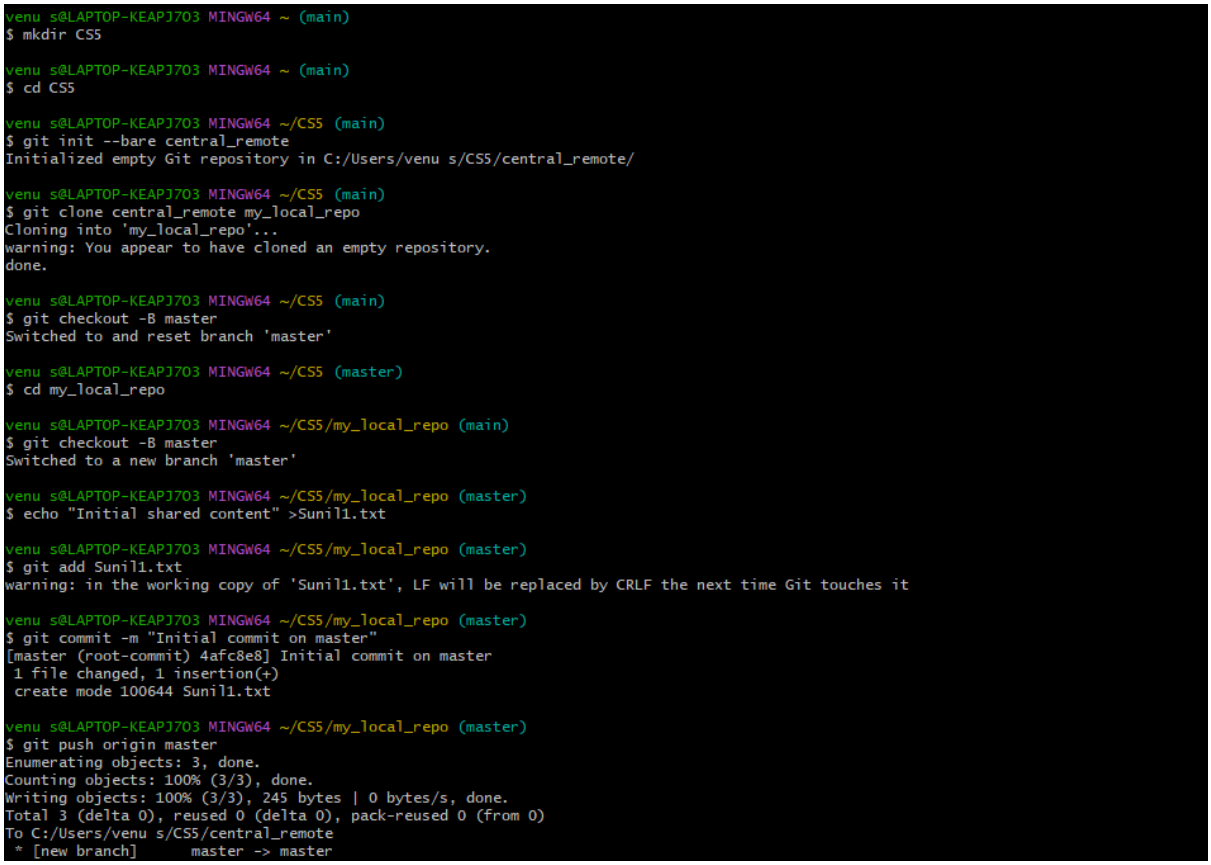
git push origin your-branch –force

Be cautious with force-pushing, especially if others are working with the same branch, as it rewrites the

commit history. Now, your local branch is rebased onto the updated remote branch. Keep in mind that

force-pushing should be done with caution, especially on shared branches, to avoid disrupting

collaborative work.



```
venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/my_local_repo (main)
$ cd ..

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5 (master)
$ git checkout -B main
Switched to and reset branch 'main'

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5 (main)
$ git clone central_remote teammate_repo
Cloning into 'teammate_repo'...
done.
warning: remote HEAD refers to nonexistent ref, unable to checkout

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5 (main)
$ cd teammate_repo

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/teammate_repo (main)
$ echo "My personal local work" >Sunil2.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/teammate_repo (main)
$ git add Sunil2.txt
warning: in the working copy of 'Sunil2.txt', LF will be replaced by CRLF the next time Git touches it

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/teammate_repo (main)
$ git commit -m "Local commit:adding text file"
[main (root-commit) eaa4e0a] Local commit:adding text file
 1 file changed, 1 insertion(+)
 create mode 100644 Sunil2.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/teammate_repo (main)
$ cd ../my_local_repo

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/my_local_repo (main)
$ git checkout  master
Switched to branch 'master'

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/my_local_repo (master)
$ echo "Update from Upstream work" >>Sunil3.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/my_local_repo (master)
$ git add Sunil3.txt
warning: in the working copy of 'Sunil3.txt', LF will be replaced by CRLF the next time Git touches it

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/my_local_repo (master)
$ git commit -m "Upstream commit: add update file"
[master 2888972] Upstream commit: add update file
 1 file changed, 1 insertion(+)
 create mode 100644 Sunil3.txt
```

```
venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/my_local_repo (master)
$ git push origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 315 bytes | 105.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To C:/Users/venu s/CS5/central_remote
   4afc8e8..2888972  master -> master

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/my_local_repo (master)
$ cd ../teammate_repo

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/teammate_repo (main)
$ git fetch origin
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 295 bytes | 16.00 KiB/s, done.
From C:/Users/venu s/CS5/central_remote
   4afc8e8..2888972  master     -> origin/master

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/teammate_repo (main)
$ git rebase origin/master
Successfully rebased and updated refs/heads/main.

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/CS5/teammate_repo (main)
$ git log --oneline --graph
* 2a44ef8 (HEAD -> main) Local commit:adding text file
* 2888972 (origin/master) Upstream commit: add update file
* 4afc8e8 Initial commit on master
```

Program 7

Git Tags and Releases:

Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository

To create a lightweight Git tag named “v1.0” for a specific commit in your local repository, you can use the following command:

git tag v1.0 <commit_hash>

Replace **<commit_hash>** with the actual hash of the commit for which you want to create the tag.

For example, if you want to tag the latest commit, you can use the following:

git tag v1.0 HEAD

This creates a lightweight tag pointing to the specified commit. Lightweight tags are simply pointers to

specific commits and contain only the commit checksum.

If you want to push the tag to a remote repository, you can use:

git push origin v1.0

This command pushes the tag named “v1.0” to to the remote repository. Keep in mind that Git tags, by

default, are not automatically pushed to remotes, so you need to explicitly push them if needed.

```
venu s@LAPTOP-KEAPJ7O3 MINGW64 ~ (main)
$ mkdir Sunil7

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~ (main)
$ cd Sunil7

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil7 (main)
$ git init
Initialized empty Git repository in C:/Users/venu s/Sunil7/.git/

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil7 (main)
$ git checkout -B master
Switched to a new branch 'master'

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil7 (master)
$ echo "Release ready Source code" > Sunil.py

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil7 (master)
$ git add Sunil.py
warning: in the working copy of 'Sunil.py', LF will be replaced by CRLF the next time Git touches it

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil7 (master)
$ git commit -m "feat: complete initial release version"
[master (root-commit) f9afdb8] feat: complete initial release version
 1 file changed, 1 insertion(+)
 create mode 100644 Sunil.py

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil7 (master)
$ git tag v1.0

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil7 (master)
$ git tag
v1.0
```

```
venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil7 (master)
$ git show v1.0
commit f9afdb8c4b7914f7a107908d4f246b6ab358652c (HEAD -> master, tag: v1.0)
Author: Sunil Kumar <sunilcs8431@gmail.com>
Date:   Wed Oct 21 19:01:58 2026 +0530

    feat: complete initial release version

diff --git a/Sunil.py b/Sunil.py
new file mode 100644
index 0000000..f02cb70
--- /dev/null
+++ b/Sunil.py
@@ -0,0 +1 @@
+Release ready Source code

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil7 (master)
$ git log --oneline --decorate -n 1
f9afdb8 (HEAD -> master, tag: v1.0) feat: complete initial release version
```

Program 8

Advanced Git Operations:

Write the command to cherry-pick a range of commits from "source-branch" to the current branch.

To cherry-pick a range of commits from “source-branch” to the current branch, you can use the following

command:

git cherry-pick <start-commit>^..<end-commit>

Replace <start-commit> and <end-commit> with the commit hashes or references that define the range of

commits you want to cherry-pick. The ^ (caret) symbol is used to exclude the starting commit itself from the range.

For example, if you want to cherry-pick the commits from commit A to commit B (excluding A) from

“source-branch” to the current branch, you would run:

git cherry-pick A^..B

After running this command, Git will apply the specified range of commits onto your current branch. If there are any conflicts, Git will pause the cherry-pick process and ask you to resolve them. After resolving conflicts, you can continue the cherry-pick with:

git cherry-pick –continue

If you encounter issues and need to abort the cherry-pick operation, you can use:

git cherry-pick –abort

```
venu s@LAPTOP-KEAPJ7O3 MINGW64 ~ (main)
$ mkdir Sunil8

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~ (main)
$ cd Sunil8

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (main)
$ git init
Initialized empty Git repository in C:/Users/venu s/Sunil8/.git/

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (main)
$ git checkout -B master
Switched to a new branch 'master'

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (master)
$ echo "Initial baseline system" > Sunil.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (master)
$ git add Sunil.txt
warning: in the working copy of 'Sunil.txt', LF will be replaced by CRLF the next time Git touches it

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (master)
$ git commit -m "Initial commit on master"
[master (root-commit) 421cd93] Initial commit on master
 1 file changed, 1 insertion(+)
 create mode 100644 Sunil.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (master)
$ git checkout -b source_branch
Switched to a new branch 'source_branch'

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ echo "Feature part 1" > part1.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ git add part1.txt
warning: in the working copy of 'part1.txt', LF will be replaced by CRLF the next time Git touches it

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ git commit -m "Feet: Add part 1"
[source_branch 0a785ff] Feet: Add part 1
 1 file changed, 1 insertion(+)
 create mode 100644 part1.txt
```

```
venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ echo "Feature part 2" > part2.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ git add part2.txt
warning: in the working copy of 'part2.txt', LF will be replaced by CRLF the next time Git touches it

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ git commit -m "Feet: Add part 2"
[source_branch f1908b7] Feet: Add part 2
 1 file changed, 1 insertion(+)
 create mode 100644 part2.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ echo "Feature part 3" > part3.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ git add part3.txt
warning: in the working copy of 'part3.txt', LF will be replaced by CRLF the next time Git touches it

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ git commit -m "Feet: Add part 3"
[source_branch 003c73c] Feet: Add part 3
 1 file changed, 1 insertion(+)
 create mode 100644 part3.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ git log --oneline
003c73c (HEAD -> source_branch) Feet: Add part 3
f1908b7 Feet: Add part 2
0a785ff Feet: Add part 1
421cd93 (master) Initial commit on master

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (source_branch)
$ git checkout master
Switched to branch 'master'
```

```
venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (master)
$ git cherry-pick 0a785ff^..003c73c
[master 07a36ba] Feet: Add part 1
 Date: Wed Oct 21 19:13:57 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 part1.txt
[master 39bc3f8] Feet: Add part 2
 Date: Wed Oct 21 19:14:35 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 part2.txt
[master 8b7cbaa] Feet: Add part 3
 Date: Wed Oct 21 19:15:06 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 part3.txt

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (master)
$ git log --oneline --graph
* 8b7cbaa (HEAD -> master) Feet: Add part 3
* 39bc3f8 Feet: Add part 2
* 07a36ba Feet: Add part 1
* 421cd93 Initial commit on master

venu s@LAPTOP-KEAPJ7O3 MINGW64 ~/Sunil8 (master)
$ ls
Sunil.txt  part1.txt  part2.txt  part3.txt
```